

Functions User Manual

Contents

Foreword	6
Functions to start	7
Function: url	8
Function: geturl	8
Function: getUrlTitle	9
Function: switchTab	9
Function: openNewTab	10
Function: clickNewTab	10
Function: closeTab	10
Function: loginUser	11
Function: loginPassword	12
Function: dummyExtraInfo	13
Function: dummyLogin	13
Function: speak	14
Function: debug	14
Basic functions	15
Function: detectGUI	16
Function: pause	16
Function: waitFor	17
Function: waitForNot	17
Function: click	18
Function: doubleClick	18
Function: JSclick	19
Function: enable	19
Function: removeAttribute	20
Function: setAttribute	20
Function: readAttribute	21
Function: setFocus	21
Function: JSinput	22

Function: keyboard	22
Function: pressEnter	23
Function: pressEscape	23
Function: pressTab	23
Function: acceptPopup	24
Function: cancelPopup	24
Function: popupKeys	25
Function: showAllPopups	25
Function: rule	26
Function: countElement	26
Function: check	27
Function: unchecked	27
Function: message	28
Function: uploadFile	28
Function: ask	29
Function: email	30
References and Data	31
Function: getReference	32
Function: setReference	32
Function: setVariable	33
Function: getData	35
Function: setData	35
Condition	36
Function: stopTest	37
Function: skipDescribe	37
Function: skipIt	38
Function: isCheck	38
Function: isExist	39
Function: isEnabled	39
Function: isVisible	40
Functions to manage an element	41
Function: switchToFrame	42
Function: getValue	42

Function: setValue	43
Function: select	44
Function: selectCount	45
Image Functions	47
Function: printscreen	48
Function: imageBaseline	49
Function: imageDifference	50
Function: imageDifferenceData	51
Epoch Date Functions	52
Function: epoch	53
Function: epochDate	53
Function: epochAddHour	54
Function: epochAddMinute	54
Function: epochAddSecond	55
Table Functions	56
Function: getTableHeader	57
Function: getTableData	57
Function: setTableData	58
Function: clickCell	58
Function: countTableRow	59
Function: searchTableData	59
HTTP Request	60
Function: httpGet	61
Function: httpPost	61
Function: httpPut	62
Function: httpDelete	62
Function: httpSearchKeyValue	63
Function: httpKeyCount	64
Function: SAML_Assertion	65
Function: SAML_Transaction	66
Function: SOAP_postData	68
Advanced functions	70
Function: callScenario	71

Function: callSuite	71
Function: startTimer	72
Function: stopTimer	72
Function: promptAI	73
Mobile functions	74
Function: phoneConnect	75
Function: phoneTap	75
Function: phoneFill	76
Function: phonePress	76
Function: phoneUrl	77
Function: phoneCapture	77
ODBC Database functions	78
Function: ODBC_Generic	79
Function: ODBC_SQLServer	80
Function: ODBC_Oracle	81
Function: ODBC_MySQL	82
Function: ODBC_ExecuteSQL	83
Function: ODBC_DataCount	84
Function: ODBC_DataValue	85

Foreword

This user manual will give you detail on the functions available in the tests and in the rules.

Tests versus Rules

In the Rules, you can use all the available functions for the tests.

However, for the rules, the name of the function is prefixed by # and ended by : (E.g.: #click:)

The parameters are separated by a comma (E.g: #click: @OPSYS_Listbox, 5, 2)

Naming convention

- Dictionary starts always with @
- Dataset starts always with #
- Rule starts always with #
- Variable start always with \$ (but in the case of a rule, it must be \$)
- Naming convention can be all in uppercase or lowercase: @URL_ACCEPTANCE
- Naming convention can be a mix @URL_Acceptance
- Naming convention can be with spaces or not: @URL_Environment Acceptance
- Functions name are case sensitive

Functions to start



Function: url

Objectives

Method to browse a website

Parameter(s)

URL	Link to the webpage to open (can be a word in the dictionary)
------------	---

Example(s):

url	@URL_OPSYS_ACC EUROPA	Start the application in acceptance
url	https://webgate.acceptance.ec.europa.eu/mwp/home?1fa	

Function: geturl

Objectives

Method to get the current url from the browser

Parameter(s)

Variable	The name of the variable to store the current url (starting with \$)
-----------------	--

Example(s):

geturl	\$currentURL	Store the current url into the variable \$currentURL
--------	--------------	--

Note: in the rule, the sign \$ for the variable must be replaced by the sign: %

Example: #geturl: %currentURL

Function: getUrlTitle

Objectives

Method to get the title of the current web page

Parameter(s)

Variable	The name of the variable to store the title (starting with \$)
-----------------	--

Example(s):

getUrlTitle	\$Title	Store the title of the current web page into the variable \$Title
-------------	---------	---

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #getUrlTitle: \$Title

Function: switchTab

Objectives

Method to switch to a specific tab on the browser

Parameter(s)

Tab position	tab position starting by 1 (0 for the last one)
---------------------	---

Example(s):

switchTab	0	Switch to the last tab
switchTab	2	Switch to the second tab

Function: **openNewTab****Objectives**

Method to open new tab on the browser (the tab will become the active one)

Note: with playwright the method is equivalent to open a new window!

Parameter(s)

Url	Url text or a word in the dictionary or a variable name starting with \$
------------	--

Example(s):

openNewTab	@URL_javadoc	Create a new tab on the browser with the link
-------------------	--------------	---

Function: **clickNewTab****Objectives**

Click to open a new tab in the browser.

Note: Contrary to the function openNewTab, the clickNewTab open a real tab in the browser

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
-------------	--

Example(s):

clickNewTab	@link_javadoc_button	Click on the button to open a new tab on the browser with the link
--------------------	----------------------	--

Function: **closeTab****Objectives**

Close the current tab and back to the first one.

Example(s):

closeTab		Close the current tab and back to the first one.
-----------------	--	--

Function: **loginUser**

Objectives

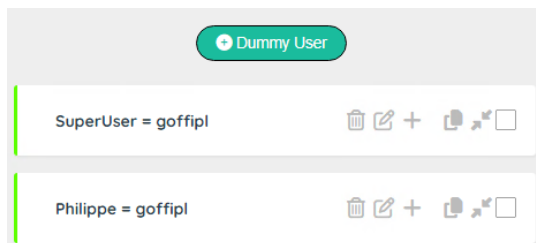
Method to login the user to an application.

Parameter(s)

Dummy user	The dummy user must be defined in the table (see: Tester User Manual)
User tag	Enter the xpath (or dictionary word) to access the user field
Submit tag	[Optional] Enter the xpath (or dictionary word) to access the submit button

Example(s):

The dummy user must be defined in the dummy user entity



loginUser	Philippe, @APP_tagLogin, @APP_tagSubmitLogin	Login the user to a screen with a submit button.
loginUser	Philippe, @APP_tagLogin	Login the user to a screen without a submit button.

Screen with a submit tag	Screen without a submit tag

Function: loginPassword

Objectives

Method to key the password to an application.

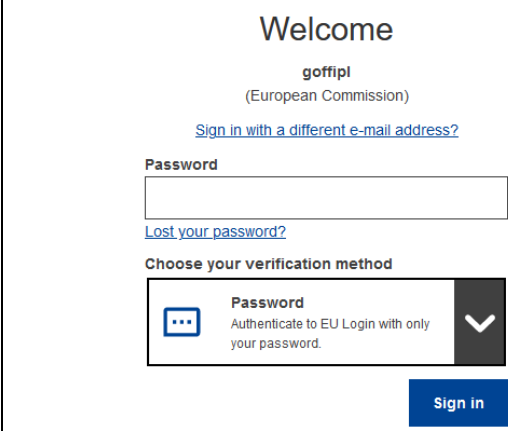
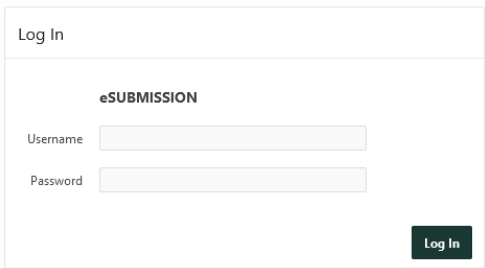
For security reason, the password is decrypted (if necessary) by the server with the information contain in the dummy entity.

Parameter(s)

Dummy user	The dummy user must be defined in the table (see: Tester User Manual)
Password tag	Enter the xpath (or dictionary word) to access the password field
Submit tag	Enter the xpath (or dictionary word) to access the submit button

Example(s):

loginUser	Philippe, @APP_tagPassword, @APP_tagSubmitPassword	Key the password to a login screen.
-----------	---	-------------------------------------

Screen with a submit tag	Screen without a submit tag
	

Function: **dummyExtraInfo****Objectives**

Method to get the extra info stored in the dummy user entity.

This information can be useful to complete a login.

It can be useful with the <ME> parameter to get information of the connected person

Parameter(s)

Dummy user	The dummy user must be defined in the table (see: Tester User Manual)
Variable	The name of the variable to store information (starting with \$)

Example(s):

dummyExtraInfo	Philippe, \$PhoneName	Get the phone name of the dummy user.
dummyExtraInfo	<ME>, \$PhoneName	Get the phone name of the connected user.

Note: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #dummyExtraInfo: Philippe, §PhoneName

Function: **dummyLogin****Objectives**

Method to get the login info stored in the dummy user entity.

This information can be useful with the <ME> parameter to get the login of the connected person.

Parameter(s)

Dummy user	The dummy user must be defined in the table (see: Tester User Manual)
Variable	The name of the variable to store information (starting with \$)

Example(s):

dummyLogin	<ME>, §MyLogin	Get the login of the connected user.
------------	----------------	--------------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #dummyLogin: <ME>, §MyLogin

Function: **speak**

Objectives

Method to text-to-speech a message

Parameter(s)

Text	Text to say (can be variable starting with \$)
-------------	--

Example(s):

speak	Hello dear tester	Say a sentence
--------------	-------------------	----------------

Function: **debug**

Objectives

Method to display more or less message to the console

Parameter(s)

Debug level	0: No Debug, 1: Important info, 2: Full detail
--------------------	--

Example(s):

debug	0	No debug message sent to the console
--------------	---	--------------------------------------

Basic functions



Function: **detectGUI****Objectives**

Method to detect the signature of an element based on generic patterns.

Patterns are generated by the AI Robot (see specific user documentation on AI Robot).

If detectGUI is successful, the variable **\$GUI** will contain the xpath to access the element

Parameter(s)

Element	Select a selector (Button, Input filed...). selector depends on the project (stored in the entity: Selector in the AI Robot)
Criteria	Enter the criteria (generally the label)
Position	Enter the position/occurrence (1 by default), \$\$ for last record or \$\$-<position>
Stop on Error	[Optional]: Stop on error (otherwise a warning is sent)

Example(s):

detectGUI	Button, Save, 1 Button, Save, 1, 0	Searching for the button 'Save'. If not found, send a warning and continue the tests
detectGUI	Button, Save, 1, 1	Searching for the button 'Save'. If not found, stop all the tests
detectGUI	Button, Save, \$\$	Searching for the last button 'Save'

Function: **pause****Objectives**

Method to wait a few seconds before continuing the next step.

Parameter(s)

Delay	Select a selector (Button, Input filed...). selector depends on the project (stored in the entity: Selector in the AI Robot)
--------------	--

Example(s):

pause	3	Wait 3 seconds
--------------	---	----------------

Function: **waitFor**

Objectives

Method to wait for the refresh of an element (to be visible).

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait	Waiting time in second(s) (default 5 sec)
Continue	What to do if the element is not ready after the waiting time: 1: Stop all the tests, 0: Continue even with an error, 2: Skip the IT

Example(s):

waitFor	@APP_Save, 5, 1	Wait 5 seconds for the button Save to be visible. If it is not the case, stop all the tests
----------------	-----------------	--

Function: **waitForNot**

Objectives

Method to wait for the element to disappear (reverse of waitFor)

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait	Waiting time in second(s) (default 5 sec)
Continue	What to do if the element is still there after the waiting time: 1: Stop all the tests, 0: Continue even with an error, 2: Skip the IT

Example(s):

waitForNot	@APP_In Progress, 15, 1	Wait 15 seconds for the text "in progress..." disappears. If it is not the case, stop all the tests
-------------------	-------------------------	---

Function: **click**

Objectives

Method to click on an element.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait after	Waiting time in second(s) after the click
Focus	1: set the focus on the element, 0: no focus on the element

Example(s):

click	@APP_Save, 3, 1	Set the focus on the button 'Save', click on it and wait 3 seconds
--------------	-----------------	--

Function: **doubleClick**

Objectives

Method to double click on an element.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait after	Waiting time in second(s) after the double click

Example(s):

doubleClick	@APP_Save, 3	Double click on the button 'Save' and wait 3 seconds
--------------------	--------------	--

Function: JSclick**Objectives**

JavaScript method to click on an element this method is more brutal force).

Can be used, if the click() is not working due to an invalid webpage status (stale)

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait after	Waiting time in second(s) after the click

Example(s):

JSclick	@APP_Save, 3	Click on the button 'Save' (CSS/Xpath in the dictionary) and wait 3 seconds
----------------	--------------	---

Function: enable**Objectives**

JavaScript method to enable an element (by removing disabled attributes)

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
-------------	--

Example(s):

enable	@APP_Save	Make the button save enabled
---------------	-----------	------------------------------

Function: **removeAttribute**

Objectives

JavaScript method to remove attribute of an element.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Attribute	Attribute to be removed

Example(s):

removeAttribute	@APP_Save, color	Remove the attribute: color
------------------------	------------------	-----------------------------

Function: **setAttribute**

Objectives

JavaScript method to remove attribute of an element.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Attribute	Attribute to assign
Value	Value of the attribute

Example(s):

setAttribute	@APP_Save, color, blue	set the attribute color: blue
---------------------	------------------------	-------------------------------

Function: readAttribute

Objectives

JavaScript method to get the value of an attribute of an element.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Attribute	Attribute to read
Variable	The name of the variable to store information (starting with \$)

Example(s):

setAttribute	@APP_Save, tagElement, \$Tag	get the attribute tag into the variable \$Tag
---------------------	------------------------------	---

Note: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #setAttribute: @APP_Save, tagElement, \$Tag

Function: setFocus

Objectives

JavaScript method to get the focus on an element.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait after	Waiting time in second(s) after the click

Example(s):

setFocus	@APP_Save, 2	Set the focus on the button save
-----------------	--------------	----------------------------------

Function: JSinput

Objectives

JavaScript method to input a value into an element.

Can be used, if the setValue() is not working due to an invalid webpage status (stale) - this method is a brutal force.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Value	Value to key in the field

Example(s):

JSinput	@APP_Name, Phil	Key 'Phil' into the field 'Name'
----------------	-----------------	----------------------------------

Function: keyboard

Objectives

JavaScript method to key a value to simulate the keyboard.

Parameter(s)

Value	Value to key in the field
--------------	---------------------------

Example(s):

JSinput	Phil	Key 'Phil' into the field 'Name'
----------------	------	----------------------------------

Function: **pressEnter**

Objectives

JavaScript method to send an Enter key

Parameter(s)

--	--

Example(s):

pressEnter		Sent an 'Enter' key
-------------------	--	---------------------

Function: **pressEscape**

Objectives

JavaScript method to send an Escape key

Parameter(s)

--	--

Example(s):

pressEscape		Sent an 'Escape' key
--------------------	--	----------------------

Function: **pressTab**

Objectives

JavaScript method to send an tab key

Parameter(s)

Number	Number of tab to send
---------------	-----------------------

Example(s):

pressTab	2	Sent 2 'tab' key
-----------------	---	------------------

Function: `acceptPopup`**Objectives**

JavaScript method to accept a JavaScript popup

Parameter(s)

--	--

Example(s):

acceptPopup	Acknowledge the popup
--------------------	-----------------------

Function: `cancelPopup`**Objectives**

JavaScript method to reject a JavaScript popup

Parameter(s)

--	--

Example(s):

cancelPopup	Cancel the popup
--------------------	------------------

Function: **popupKeys**

Objectives

Execute a powershell process to sent keys to a windows popup.

For instance: sent the keys "{TAB}{TAB}{TAB}{TAB}{TAB}{ENTER}" to the certificate windows

Parameter(s)

windowTitle	Title of the popup windows (you can use the function: showAllPopups() to identify the open popup windows)
sendkeys	A string with the keys to send (E.g.: {ENTER})

Example(s):

popupKeys	My Certificate, {TAB} {TAB} {ENTER}	Send keys to the popup "My Certificate"
------------------	-------------------------------------	---

Function: **showAllPopups**

Objectives

Execute a powershell process to display all the popup titles

Parameter(s)

--	--

Example(s):

showAllPopups		Show all popup windows
----------------------	--	------------------------

Step	[5] Show all popups open
Info	1: Settings
Info	2: Privacy error - Chromium
Info	3: frontend - Google Chrome
Info	4: ? robot.library.js - robotv3 - Visual Studio Code
Info	5: Sopra Steria Pulse - Session 1 Sopra Steria philippe
Info	6: 2 Reminder(s)
Info	7: JamPostMessageWindow

Function: **rule**

Objectives

Method to call a rule

Parameter(s)

Rule	Name of the rule
Parameter 1	Parameter 1 for the rule (will be represented by the variable \$P1)
Parameter 2	Parameter 2 for the rule (will be represented by the variable \$P2)

Example(s):

Rule	Login ECAS, \$DummyUser	Rule to login with a dummy user
-------------	-------------------------	---------------------------------

Function: **countElement**

Objectives

Method to call a rule

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	The name of the variable to store information (starting with \$)

Example(s):

countElement	@APP_section, \$NbSection	Count the number of sections on the screen
---------------------	---------------------------	--

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #countElement: @APP_section, \$NbSection

Function: **check**

Objectives

Method to check a checkbox (if necessary)

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait after	Waiting time in second(s) after the check

Example(s):

check	@APP_approve, 5	Check the approve checkbox
--------------	-----------------	----------------------------

Function: **unchecked**

Objectives

Method to unchecked a checkbox (if necessary)

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Wait after	Waiting time in second(s) after the unchecked

Example(s):

unchecked	@APP_approve, 5	Uncheck the approve checkbox
------------------	-----------------	------------------------------

Function: **message****Objectives**

Method to write a message in the log file

Parameter(s)

Message	Message to display in the log file
Category	Info, Message, Warning or Error

Example(s):

message	Test successful, Info	Write a message
----------------	-----------------------	-----------------

Function: **uploadFile****Objectives**

Method to upload a file from the repository uploads of your project.

The repository is managed by the Administrator.

Note: Prior to use the function, the ADMIN must store the document in the Upload section of the project.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
File	Name of the file (without a path)

Example(s):

uploadFile	@APP_Upload, price.pdf	Upload the file price.pdf
-------------------	------------------------	---------------------------

Function: ask

Objectives

Method to display a popup window to invite the user to enter a value

Parameter(s)

Message	The message to display to the user
Default value	[Optional] Default value
Variable	Name of the variable to store the value (by default \$Ask)
Timeout	Timeout in seconds (default 30 seconds)

Example(s):

ask	Please enter the environment, ACC, \$Env, 120	Ask the user to provide the name of the environment. After 120 seconds the variable \$Env is filled with ACC
------------	---	--

Note: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #ask: Enter the environment, ACC, §Env, 20


Robot

Please enter the environment

Enter a value and submit...

Submit

Auto-submitting in 105s...

Function: **email**

Objectives

Method to send an email with attachment(s) (optional).

The originator of the message is managed by the Administrator and is stored in the project parameters:

Email Host - smtpmail.cec.eu.int:25

Email From - automated-notifications@nomail.ec.europa.eu (AutoTest)

Parameter(s)

Email To	Recipient (comma separated for multiple people)
Subject	subject of the message
Body	body of the message (keywords: <BLUE><RED>.. <BOLD><ITALIC><NORMAL> Body can contain html tag (E.g.: <table>, <body>, <tr>, <td>....)
Attachment	[optional] Full path name of the attachment(s) - use ';' as a separator

Example(s):

email	\$To, \$Subject, \$Body, \$Attachment	Send an email
--------------	---------------------------------------	---------------

Example of body for a sanity check with an error in one of the environment.

<BOLD><RED>Error detected in \$Environment**<NORMAL><NORMAL>**

Note: a tag **<NORMAL>** must be added after the tags **<BLUE><RED>..**<BOLD><ITALIC>****

In order to use the function, the ADMIN must create two parameters at the project level

Parameter name	Example
Email Host	smtpmail.cec.eu.int:25
Email From	automated-notifications@nomail.ec.europa.eu (AutoTest)

References and Data



Function: **getReference****Objectives**

Method to get a reference by Code. The reference is used by the Robot to exchange (read/write) data between the scenarios.

Parameter(s)

Code	Code of the reference
Variable	Name of the variable to store the value (starting with \$)

Example(s):

getReference	Dataset, \$Dataset	Get the value of the dataset in the reference
--------------	--------------------	---

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #getReference: Dataset, \$Dataset

Function: **setReference****Objectives**

Method to get a reference. The reference is used by the Robot to exchange (read/write) data between the scenarios.

Parameter(s)

Code	Code of the reference
Value	Value of the reference (can be a variable starting with \$)
Comment	[]Optional If Comment is empty, the value will not be overridden

Example(s):

getReference	Dataset, \$Dataset	Get the value of the dataset in the reference
--------------	--------------------	---

Function: **setVariable****Objectives**

Method to set a variable.

Parameter(s)

Variable	Name of the variable (starting with \$)
Value	Enter a value or <EMPTY> or an expression (must start with =)

Example(s):

setVariable	\$Test, A simple test	Set a text into the variable
setVariable	\$Test, = 1 + 1	Set 2 into the variable
setVariable	\$Test, <EMPTY>	Reset the variable
setVariable	\$Test, <TODAY>	Use a keyword to get the current date

Note 1: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #setVariable: §Test, A simple test

Note 2: Special keywords are:

With **nn** as a numeric value

<TODAY>, <TODAY+nn>, <TODAY-nn> to get the current date + or – days(s) – Format: DD/MM/YYYY

<NOW>, <NOW+nn>, <NOW-nn> to get the current date + or – day(s) - Format: DD/MM/YYYY HH:mm

<YEAR>, <YEAR+nn>, <YEAR-nn> to get the current year + or – year(s) – Format: YYYY

<MONTH>, <MONTH+nn>, <MONTH-nn> to get the current month + or – month(s) – Format: MM

<DAY>, <DAY+nn>, <DAY-nn> to get the current day + or – days(s) – Format: DD

<HOURS>, <HOUR+nn>, <HOUR-nn> to get the current hour + or – hour(s) – Format: HH

<SEQUENCE> to get a unique number – Format: YYYYMMDD_hmmss

Manipulation on a text

As you can use standard JavaScript in the expression of a variable (expression starts by =),

You can very easily manipulate a text:

In this example, we assume that the variable \$Message contains "hello your ID is 1234567890"

Return the last 10 characters	\$Test, =\$Message.slice(-10)	1234567890
Return the first 5 characters	\$Test, = \$Message.slice(0, 5)	Hello
Return the text inside out starting from the 12rd character, with a length of 2	\$Test, =\$Message.substr(11, 2) As the string starts at 0, 12rd char is at the position 11	ID
Return true or false if a short text is contained within a long string	\$Test, =\$Message.includes('hello')	True
Replace a part of the text	\$Test, =\$Message.replace('hello', 'Hi,')	Hi, your ID is 1234567890
Replace all by "	\$Test, =\$Message.replace(/<br\s*\/?>/gi, " ")	All the tag will be replaced (if any)
Remove leading and trailing spaces from a word	\$Test, =\$Message.trim()	

Function: **getData****Objectives**

Method to get a data by its code. Data are store in a dataset and are managed by the Tester.

Parameter(s)

Code	Code of the data Code of the data (format: #<dataset>_<data>)
Variable	Name of the variable to store the value (starting with \$)

Example(s):

getData	#Data_Dataset, \$Dataset	Get a value from the dataset
---------	--------------------------	------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #getData: #DATA_DirectLink, \$DirectLink

Function: **setData****Objectives**

Method to set a data with a value. Data are store in a dataset and are managed by the Tester.

Parameter(s)

Code	Code of the data Code of the data (format: #<dataset>_<data>)
Value	Name of the variable to store the value (starting with \$)
Comment	Comment for the data

Example(s):

setData	#Data_Dataset, SEA-2023, Contract SEA-2023	Set a value in the dataset
---------	--	----------------------------

Condition



Function: **stopTest****Objectives**

Method to stop all the tests if a condition is true.

Parameter(s)

Condition	Any valid JavaScript expression that returns true or false (or a variable)
Message	Message to display when the condition is true

Example(s):

stopTest	\$Error == 1, Error detected stop the tests	Error detection
----------	---	-----------------

Note: Condition is a JavaScript expression so: equal is ==, not equal is !=

Function: **skipDescribe****Objectives**

Method to skip the Describe section if the expression is true.

Parameter(s)

Condition	Any valid JavaScript expression that returns true or false (or a variable)
Message	Message to display when the condition is true

Example(s):

skipDescribe	\$Action != 'Document', Skip the document section	Skip a Describe section
--------------	---	-------------------------

Note: Condition is a JavaScript expression so: equal is ==, not equal is !=

Function: **skiplt**

Objectives

Method to skip the IT section if the expression is true.

Parameter(s)

Condition	Any valid JavaScript expression that returns true or false (or a variable)
Message	Message to display when the condition is true

Example(s):

skiplt	\$Exits == 0, Skip the test, no field detected !	Skip a IT section
---------------	--	-------------------

Note: Condition is a JavaScript expression

Function: **isCheck**

Objectives

Method to detect if an element is checked. 1: if element is checked, otherwise 0

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	Name of the variable to store the result: 1 or 0 (starting with \$)
Wait for element	Waiting time in second(s) for the element to be ready

Example(s):

isCheck	@APP_checkbox, \$Agree, 5	Check if the checkbox is checked
----------------	---------------------------	----------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #isCheck: @APP_checkbox, 5, \$Agree

Function: **isExist**

Objectives

Method to detect if an element exists. 1: if element exists, otherwise 0

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	Name of the variable to store the result: 1 or 0 (starting with \$)
Wait for element	Waiting time in second(s) for the element to be ready

Example(s):

isExist	@APP_checkbox, \$Exist, 5	Check if the checkbox exists
---------	---------------------------	------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #isExist: @APP_checkbox, 5, §Exist

Function: **isEnabled**

Objectives

Method to detect if an element is enabled. 1: if element is enabled, otherwise 0

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	Name of the variable to store the result: 1 or 0 (starting with \$)
Wait for element	Waiting time in second(s) for the element to be ready

Example(s):

isEnabled	@APP_checkbox, §Enabled, 5	Check if the checkbox is enabled
-----------	----------------------------	----------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #isEnabled: @APP_checkbox, 5, §Enabled

Function: isVisible

Objectives

Method to detect if an element is visible. 1: if element is visible, otherwise 0

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	Name of the variable to store the result: 1 or 0 (starting with \$)
Wait for element	Waiting time in second(s) for the element to be ready

Example(s):

isVisible	@APP_checkbox, \$Visible, 5	Check if the checkbox is visible
-----------	-----------------------------	----------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: §

Example: #isVisible: @APP_checkbox, 5, §Visible

Functions to manage an element



Function: **switchToFrame**

Objectives

Method to get a value from a field.

Note: the Robot is able to manage automatically the frame and the iFrame. This function is there only in case you need to perform a special operation.

Parameter(s)

Frame	Frame 0 is the default one
--------------	----------------------------

Example(s):

switchToFrame	1	Switch to frame 1
---------------	---	-------------------

Function: **getValue**

Objectives

Method to get a value from a field.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	Name of the variable to store the result (starting with \$)

Example(s):

getValue	@APP_Name, \$Name	Get the value of the field Name
----------	-------------------	---------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: &

Example: #getValue: @APP_Name, &Name

Function: **setValue****Objectives**

Method to set a value into a field.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Value	Value to key (by default closed by TAB), or use <ENTER> or a variable
Delay	[Optional] Delay in second(s) before the <TAB> or <ENTER> or after keying the value – Very useful, when you have to wait for the construction of a list

Example(s):

setValue	@APP_Name, \$Name	set the value in the field Name
setValue	@APP_Decision, \$Decision<TAB>, 3	Enter a decision, wait for 3 sec and key a Tab

If the value is <N/A> or <EMPTY> the function will not be executed but will return with the status success.

In the logfile, the info will be <N/A> or <EMPTY> (Skipped!)

Step	[43] Enter the abbreviation
Info	<N/A> (Skipped!)

Function: **select**

Objectives

Method to select a value from a list.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Value	Value to key or a variable or a position (@<position>)
Wait after	Waiting time in second(s) after the click (default 2 sec)

Example(s):

select	@APP_Country, \$Country, 3	Select a country
--------	----------------------------	------------------

Note 1: Only the standard html <select><option> is recognized.

OBSOLETE

Note 2: Value is by default searched with a contains (approximate matching).

For compliance reason, you can use <*> but it has no impact!

To force an exact match: use = as the first character - Example: =Dupond

To get a specific option: use @<position> - Example @2 to get the second option

<Aa> is not a valid option. Value is always case sensitive.

Note 3: \$Value contains the item selected from the list (useful when using position: E.g. @1)

If the value is <N/A> or <EMPTY> the function will not be executed but will return with the status success.

In the logfile, the info will be <N/A> or <EMPTY> (Skipped!)

Step	[43] Enter the abbreviation
Info	<N/A> (Skipped!)

Function: selectCount**Objectives**

Method to count the number of values in a list.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	Name of the variable to store the result: 1 or 0 (starting with \$)

Example(s):

selectCount	@APP_Country, \$Countries	Count the number of countries
-------------	---------------------------	-------------------------------

Note 1: Only the standard html <select><option> is recognized.

Image Functions



Function: **printscreen**

Objectives

Method to take a print screen and store the image on a slot (up to 5 slots available)

Parameter(s)

slot	From 1 to 5
Capture mode	0 : Normal view, 1 : Full page

Example(s):

printscreen	1	0	Take a normal print screen and store it in the slot 1
printscreen	1	1	Take a full print screen and store it in the slot 1

Function: **imageBaseline****Objectives**

Method to take a print screen and store it as the baseline (to create or refresh the baseline).

Parameter(s)

Baseline Name	Name the base line to use
Print screen Slot	From 1 to 5
Mask 1 → 5	You can add up to 5 locator to exclude for the print screen (you can use a word from the dictionary)

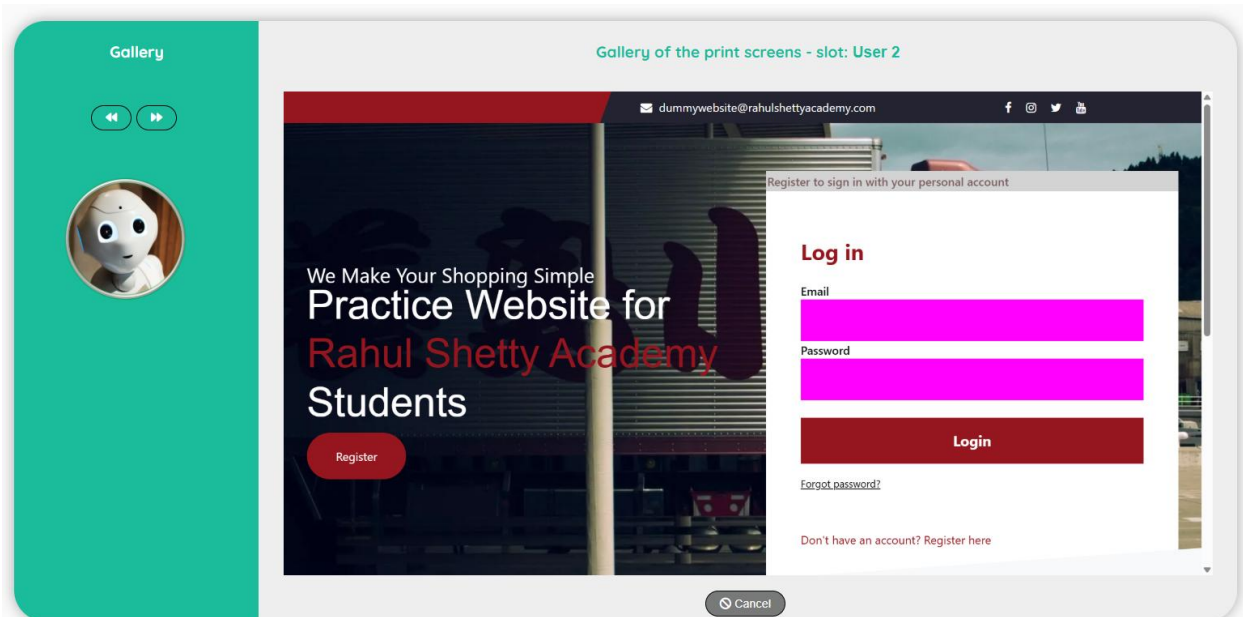
Example(s):

imageBaseline	Orders	1	Take a print screen and store the image as a baseline. The baseline is visible in the slot 1.
----------------------	--------	---	--

In this example, we used a mask on the login and the password.

You can see that the boundaries of the elements are replaced by a pink box.

The color is defined by Playwright and it's a color that will help for the comparison (used by the function **imageDifference**)



Function: `imageDifference`

Objectives

Method to take a print screen compare the image with a baseline

Parameter(s)

Baseline Name	Name the base line to use
Print screen Slot	From 1 to 5
Baseline Slot	Optional: From 1 to 5 to store the baseline image in a slot
Mask 1 → 5	You can add up to 5 locator to exclude for the print screen (you can use a word from the dictionary)

Example(s):

imageDifference	Orders	1	-1	Compare the baseline "Orders" with a fresh print screen (stored in the slot 1)
imageDifference	Orders	1	2	Compare the baseline "Orders" with a fresh print screen (stored in the slot 1) and store the baseline image in the slot 2

Note 1: if the baseline picture doesn't exit, the function create the image and stop.

You can also use the function `imageBaseline()` to create/refresh the baseline.

Note 2: There is a known bug and the images are not directly visible in the print screen slot. A refresh of the browser is required (with a login/password) to display the images.

Note 3: If you want accurate differences, be sure you use the same mask(s) as in the function `imageBaseline`.

Function: `imageDifferenceData`

Objectives

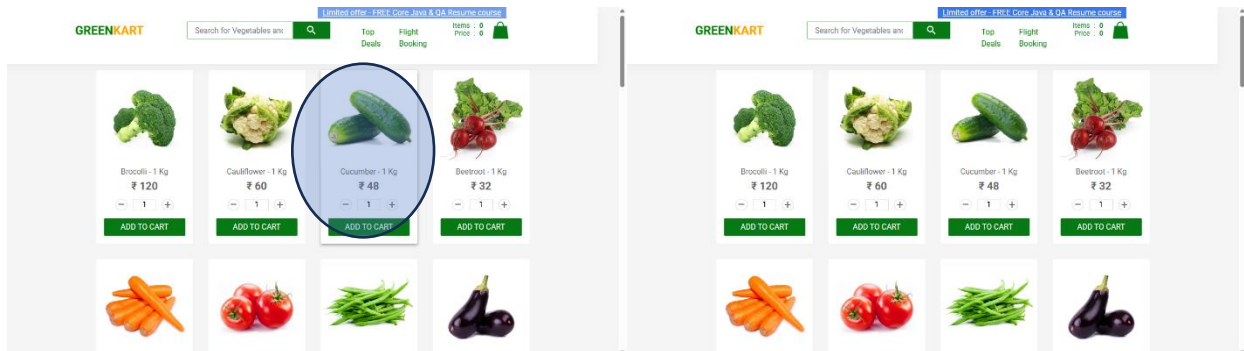
Method to store an attribute on an image difference in a variable after a call of the function `imageDifference()`

Parameter(s)

Parameter	Select a value in the list: Similarity, Difference, Different pixels, Image dimensions
Variable	Name of the variable

Example(s):

<code>imageDifferenceData</code>	Similarity	<code>\$Similarity</code>	Similarity: 98.54%
<code>imageDifferenceData</code>	Difference	<code>\$Difference</code>	Difference: 1.46%
<code>imageDifferenceData</code>	Different pixels	<code>\$Pixel</code>	Different pixels: 13489 out of 921600
<code>imageDifferenceData</code>	Image dimensions	<code>\$Size</code>	Image dimensions: 1280x720



In this example, the cucumbers are bigger on the left picture due to a mouse over.

Note: you can use the functions `imageDifference()` and `imageDifferenceData()` to detect a change in the user interface and send a message to the responsible of the documentation for instance.

Epoch Date Functions



Function: **epoch**

Objectives

Method to get a date converted into epoch (Unix) date and time.

Parameter(s)

Date	A date in any valid format
Format	Any valid format (E.g.: 'DD/MM/YYYY HH:mm:ss')
Variable	Name of the variable to store the result (starting with \$)

Example(s):

epoch	22/05/2024, DD/MM/YYYY, \$EpochDate	Convert a date
-------	-------------------------------------	----------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #epoch: 22/05/2024, DD/MM/YYYY, \$EpochDate

Function: **epochDate**

Objectives

Method to convert an epoch date into a date.

Parameter(s)

Epoch Date	An epoch date
Format	Any valid format (E.g.: 'DD/MM/YYYY HH:mm:ss')
Variable	Name of the variable to store the result (starting with \$)

Example(s):

epochDate	\$EpochDate, DD/MM/YYYY, \$Date	Convert an epoch date
-----------	---------------------------------	-----------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #epochDate: \$EpochDate, DD/MM/YYYY, \$Date

Function: **epochAddHour****Objectives**

Method to get a date + hour(s) converted into epoch (Unix) date and time.

Parameter(s)

Date	A date in any valid format or NOW for the current date time
Format	Any valid format (E.g.: 'DD/MM/YYYY HH:mm:ss')
Hour	Number of hour(s) to add to the date
Variable	Name of the variable to store the result (starting with \$)

Example(s):

epochAddHour	NOW, DD/MM/YYYY, 2, \$Date	Add 2 hours and convert in epoch
---------------------	----------------------------	----------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #epochAddHour: NOW, DD/MM/YYYY, 2, \$Date

Function: **epochAddMinute****Objectives**

Method to get a date + minute(s) converted into epoch (Unix) date and time.

Parameter(s)

Date	A date in any valid format or NOW for the current date time
Format	Any valid format (E.g.: 'DD/MM/YYYY HH:mm:ss')
Minute	Number of minute(s) to add to the date
Variable	Name of the variable to store the result (starting with \$)

Example(s):

epochAddMinute	NOW, DD/MM/YYYY, 10, \$Date	Add 10 seconds and convert in epoch
-----------------------	-----------------------------	-------------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #epochAddMinute: NOW, DD/MM/YYYY, 10, \$Date

Function: **epochAddSecond****Objectives**

Method to get a date + second(s) converted into epoch (Unix) date and time.

Parameter(s)

Date	A date in any valid format or NOW for the current date time
Format	Any valid format (E.g.: 'DD/MM/YYYY HH:mm:ss')
Second	Number of second(s) to add to the date
Variable	Name of the variable to store the result (starting with \$)

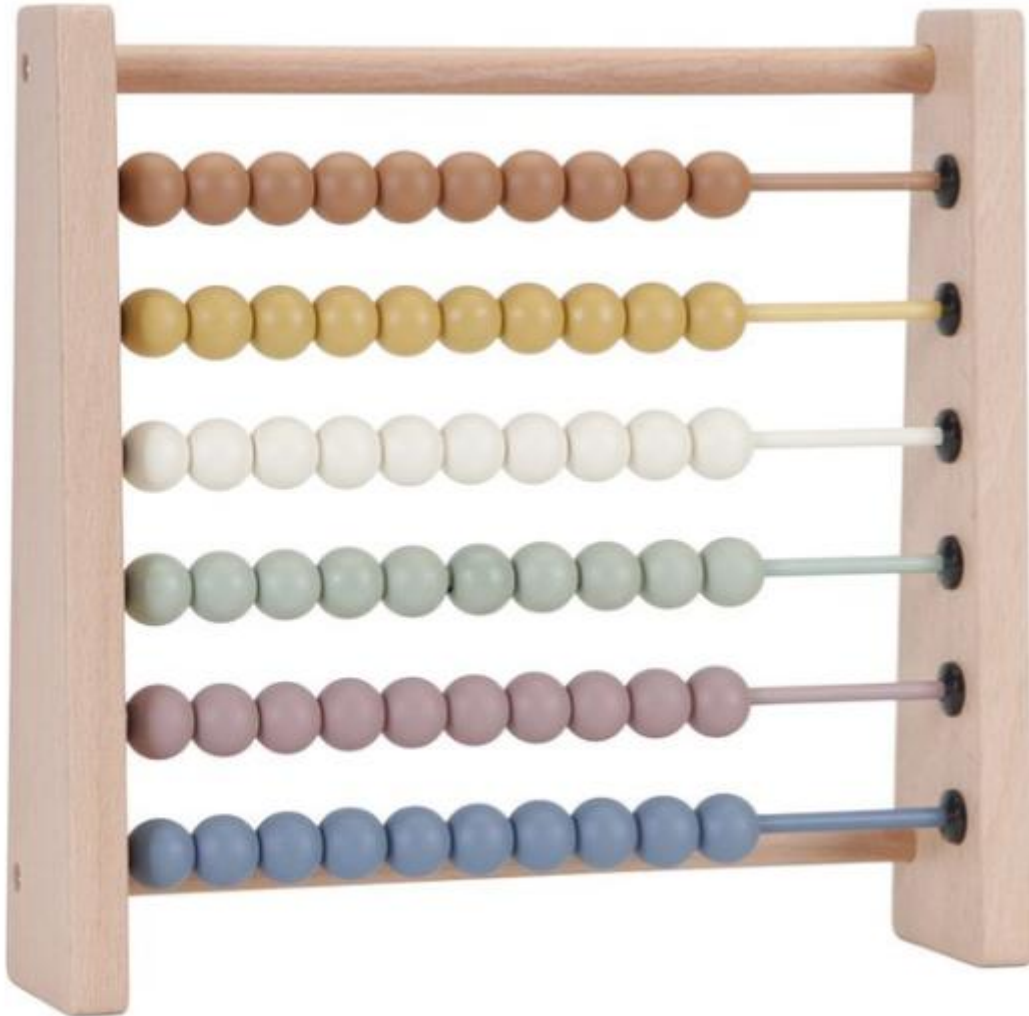
Example(s):

epochAddSecond	NOW, DD/MM/YYYY, 5, \$Date	Add 5 minutes and convert in epoch
-----------------------	----------------------------	------------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #epochAddSecond: NOW, DD/MM/YYYY, 5, \$Date

Table Functions



Function: `getTableHeader`

Objectives

Method to get a header from a table.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Row	The row position in the table
Column	The column position in the table
Variable	Name of the variable to store the result (starting with \$)

Example(s):

<code>getTableHeader</code>	<code>@APP_Amount, 1, 3, \$AmountID</code>	Get the value of the header (1,3)
-----------------------------	--	-----------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: `#getTableHeader: @APP_Amount, 1, 3, $AmountID`

Function: `getTableData`

Objectives

Method to get a value from a cell of a table.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Row	The row position in the table
Column	The column position in the table
Variable	Name of the variable to store the result (starting with \$)

Example(s):

<code>getTableData</code>	<code>@APP_Amount, 1, 3, \$Amount</code>	Get the value of the cell (1,3)
---------------------------	--	---------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: `#getTableData: @APP_Amount, 1, 3, $Amount`

Function: **setTableData****Objectives**

Method to set a value into a cell of a table.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Row	The row position in the table
Column	The column position in the table
Value	Value or variable (starting with \$)

Example(s):

setTableData	@APP_Amount, 1, 3, \$Amount	set the amount into the cell (1,3)
--------------	-----------------------------	------------------------------------

Function: **clickCell****Objectives**

Method click on a cell of a table.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Row	The row position in the table
Column	The column position in the table
Delay	Duration (in second(s) after the click)

Example(s):

clickCell	@APP_Amount, 1, 3, 5	Click in the cell (1,3)
-----------	----------------------	-------------------------

Function: **countTableRow****Objectives**

Method count the row(s) of a table.

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Variable	Name of the variable to store the result (starting with \$)

Example(s):

countTableRow	@APP_Table, \$Row	Count the number of row(s) of a table
----------------------	-------------------	---------------------------------------

Note: in the rule, the sign \$ for the variable must be replaced by the sign: \$

Example: #countTableRow: @APP_Table, \$Row

Function: **searchTableData****Objectives**

Method search for a value in a table at a specific column.

Result is stored in the variable \$Row (-1 if not found).

Parameter(s)

Name	Xpath or dictionary word (starting with @) of the element or \$GUI
Column	The column position in the table
Search Value	Value to search in the table
Occurrence	Occurrence of the search (default 1)

Example(s):

searchTableData	@APP_Table, 3, Belgium, 1	Search for Belgium in a table
------------------------	---------------------------	-------------------------------

HTTP Request



Function: httpGet**Objectives**

Perform a http request using the GET method.

The result will be stored in the httpdata with the code: "httpget"

Parameter(s)

Url	Link to the REST API (can be a variable or a word in the dictionary)
Token	Optional: can be N/A or a variable or a text or the name of a .p12 certificate
Key	Optional: only if a .p12 certificate is provided
Code	Code to store the http result (by default httpGet)

Example(s):

httpGet	Get all the licenses using REST API	http://localhost:5000/api/license	<N/A>	<N/A>
httpGet	API with a .p12 certificate	http://localhost:5000/api/license	\$Token	\$Key

Note 1: to get the result use the functions httpSearchKeyValue and httpKeyCount (see below)

Note 2: The status of the transaction is stored in the variable \$HttpStatus

Note 3: the certificate .p12 must be uploaded on the server by the Administrator

Function: httpPost**Objectives**

Perform a http request using the POST method.

The result will be stored in the httpdata with the code: "httppost"

Parameter(s)

Url	Link to the REST API (can be a variable or a word in the dictionary)
Body	JSON string to define the body parameters (can be a variable or a dataset)
Token	Optional: can be N/A or a variable or a text (or empty)
Key	Optional: only if a .p12 certificate is provided
Code	Code to store the http result (by default httpPost)

Example(s):

httpPost	Get a Natural word	@URL_API_Natural	{"code": "@Reject_the", "language": "EN", "active": 1}	<N/A>	<N/A>
httpPost	API with a .p12 certificate	@URL_API_Cert	{"code": "ABC"}	Cert.p12	\$Key

Note 1: Use only double quote in the Body expression.

Note 2: The status of the transaction is stored in the variable \$HttpStatus

Note 3: to get the result use the functions httpSearchKeyValue and httpKeyCount (see below)

Note 4: the certificate .p12 must be uploaded on the server by the Administrator

Function: **httpPut****Objectives**

Perform a http request using the PUT method.

The result will be stored in the httpdata with the code: "httpput"

Parameter(s)

Url	Link to the REST API (can be a variable or a word in the dictionary)
Body	JSON string to define the body parameters (can be a variable)
Token	Optional: can be N/A or a variable or a text (or empty)
Code	Code to store the http result (by default httpPut)

Example(s):

httpPut	Update a user	@URL_API_User	{"userId": 1, "lastname": "Dupond", "firstname": "Tom"}	N/A
----------------	---------------	---------------	---	-----

Note 1: Use only double quote in the Body expression.

Note 2: The status of the transaction is stored in the variable \$HttpStatus

Note 3: to get the result use the functions httpSearchKeyValue and httpKeyCount (see below)

Function: **httpDelete****Objectives**

Perform a http request using the DELETE method.

The result will be stored in the httpdata with the code: "httpdelete"

Parameter(s)

Url	Link to the REST API (can be a variable or a word in the dictionary)
Token	Optional: can be N/A or a variable or a text (or empty)
Code	Code to store the http result (by default httpDelete)

Example(s):

httpDelete	Delete a product	http://localhost:5000/api/product/10	\$Token
-------------------	------------------	--------------------------------------	---------

The status of the transaction is stored in the variable \$HttpStatus

Function: [httpSearchKeyValue](#)

Objectives

Get data from a previous httpGet, httpPost, httpPut, ... or SAMLTransaction functions

Parameter(s)

Code	Code of the http data
Key	Key in the result (<ALL> for a dump of the result)
Key Occurrence	Position of the key in the result (1 by default)
Parent Key	Key of the parent (Granparent) in the result (<N/A> by default)
Parent Occurrence	Position of the parent (Granparent) in the result (1 by default)
Variable	Name of the variable to store the result (-1 if not found with \$Error = 1) or a Boolean if a search value is used
Operation	Equal or Contains
Search value	(optional) <N/A> or a value to search in the result

Example(s) of httpSearchKeyValue:

Code	Key	Occ	Parent	Occ	Variable	Op	search	Comment
httpget	IDCard	1	<N/A>	1	\$IDCard			No parent
httpget	IDCard	1	IDCards	2	\$IDCard2			With parent
Httpget	<ALL>							Dump of the result in the log
Httpget	CardNr	1	IDCards	1	\$Exist	contains	591286124036	True

Example, of the http data result

```
{
  "success":1,
  "data":
    <IDCards>
      <IDCard Type="195">
        <CardNr>591286124036</CardNr>
        <ExpiryDate><Century>20</Century><Year>16</Year><Month>03</Month><Day>02</Day></ExpiryDate>
      </IDCard>
      <IDCard Type="196">
        <CardNr>123286124036</CardNr>
        <ExpiryDate><Century>20</Century><Year>26</Year><Month>03</Month><Day>02</Day></ExpiryDate>
      </IDCard>
    </IDCards>
}
```

Function: **httpKeyCount**

Objectives

Count the number of occurrences of a key in the http result data

Parameter(s)

Code	Name of the element (or a variable)
Search key	Select a value in the list: Equal or Contains
Parent key	The value to search (default <N/A>)
Variable	Name of the variable to store the result (-1 if not found and \$Error = 1)

Example(s):

httpKeyCount	httpget	FamilyMember	<N/A>	\$httpCount	Return 4
httpKeyCount	httpget	Name	FamilyMembers	\$httpCount	Return 4

In the example, the result of the httpdata is:

```
success: 1,
data:
  <FamilyMembers>
    <FamilyMember>
      <FamilyRole><Label>Epouse</Label></FamilyRole>
      <Name><LastName>Hubens</LastName><FirstName>Christianne</FirstName></Name>
    </FamilyMember>
    <FamilyMember>
      <FamilyRole><Label>Fille</Label></FamilyRole>
      <Name><LastName>Sermon</LastName><FirstName>Annelies</FirstName></Name>
    </FamilyMember>
    <FamilyMember>
      <FamilyRole><Label>Fils</Label></FamilyRole>
      <Name><LastName>Sermon</LastName><FirstName>Ronald</FirstName></Name>
    </FamilyMember>
    <FamilyMember>
      <FamilyRole><Label>Petit-fils</Label></FamilyRole>
      <Name><LastName>Dupuis</LastName><FirstName>Tim</FirstName></Name>
    </FamilyMember>
  </FamilyMembers>
```

Now, you can use the httpSearchKeyValue () to get all the items of the search via a Loop.

Put a condition: \$HttpCount > 0 and set the occurrence of the key with \$Loop

Function: SAML_Assertion

Objectives: Create a SAML Assertion

Parameter(s)

Certificate Dataset	Name of the dataset to use for the certificate
Assertion URL	URL for the assertion
Assertion Dataset	Name of the dataset to use for the assertion

Example(s):

SAML_Assertion	#SAML_Certificate	/v1.0/assertions/	#AB_Policy
-----------------------	-------------------	-------------------	------------

Example of Certificate Dataset:

#SAML_Certificate_url	https://myserver.be	URL of the certificate
#SAML_Certificate_name	myCertificate.p12	Name of the certificate
#SAML_Certificate_key	Key123	Secret key of the certificate

Note 1: the certificate must be uploaded by the Administrator on the Robot server

Note 2: You can use the certificates: .p12, .pfx or .crt (with .key)

Note 3: It's very important to respect the data set code: _url, _name and _key (case sensitive), for the name of the dataset header, there is no restriction, you can use whatever you want!

Example of Assertion Dataset:

#AB_Policy_policy	AB_POLICY	Policy name
#AB_Policy_certificate	Pub1234cbg789...	Public key or <publickey>
#AB_Policy_requestId	<requestID>	Or enter your own request ID

Note 1: you need to check the requirements of your SAML assertion to define the correct dataset

Note 2: <publickey> (non sensitive) is not working with all the certificates, if you have an error, you have to provide manually the public key!

Note3: <requested> (non sensitive) is used to generate a request with the format: REQ-<date now>

Function: **SAML_Transaction**

Objectives: Create a SAML Transaction.

The result of the transaction will be stored in the http data with the code equal to the name provided for the assertion dataset.

To get the http data result use the functions `httpSearchKeyValue` and `httpKeyCount`

Parameter(s)

Certificate Dataset	Name of the dataset to use for the certificate
Assertion Dataset	Name of the dataset to use for the assertion
Transaction URL	URL for the transaction
Transaction Dataset	Name of the dataset to use for the transaction

Example(s):

SAML_Transaction	#SAML_Certificate	#AB_Policy	/v1.0/transactions/	#ZZ99
-------------------------	-------------------	------------	---------------------	-------

Example of Certificate Dataset:

#SAML_Certificate_url	https://myserver.be	URL of the certificate
#SAML_Certificate_name	myCertificate.p12	Name of the certificate
#SAML_Certificate_key	Key123	Secret key of the certificate

Note 1: the certificate must be uploaded by the Administrator on the Robot server

Note 2: You can use the certificates: .p12, .pfx or .crt (with .key)

Note 3: It's very important to respect the data set code: `_url`, `_name` and `_key` (case sensitive), for the name of the dataset header, there is no restriction, you can use whatever you want!

Example of Assertion Dataset:

#AB_Policy_policy	AB_POLICY	Policy name
#AB_Policy_certificate	Pub1234cbg789...	Public key or <publickey>
#AB_Policy_requestId	<requestID>	or enter your own request ID

Note 1: you need to check the requirements of your SAML assertion to define the correct dataset

Note 2: <publickey> (non sensitive) is not working with all the certificates, if you have an error, you have to provide manually the public key!

Note3: <requested> (non sensitive) is used to generate a request with the format: REQ-<date now>

Example of Transaction Dataset:

#ZZ99_txMessage	%ZZ99 ABCD00070123456	txMessage
#ZZ99_signed	00	signed
#ZZ99_requestId	<requestID>	or enter your own request ID

Note 1: you need to check the requirements of your SAML transaction to define the correct dataset

Note2: <requested> (non sensitive) is used to generate a request with the format: REQ-<date now>

Function: SOAP_postData

Objectives: Create a SOAP POST http request.

The result of the transaction will be stored in the http data with the code equal to the name provided for the assertion dataset.

To get the http data result use the functions httpSearchKeyValue and httpKeyCount

Parameter(s)

Soap URL	URL for the SOAP endpoint
Soap Dataset	Name of the dataset to use for the Soap request

Example(s):

SOAP_postData	http://server/services/PersonEndpoint	#getPerson
----------------------	---------------------------------------	------------

Example of Certificate Dataset:

#getPerson_soapenv	http://schemas.org/soap/envelope/	Soap envelope
#getPerson_core	http://www.server/core	Soap core
#getPerson_header	<!-- No header -->	No header in this example
#getPerson_body	<pre><core:getPerson> <!-- person identifier --> <core:nationalNumber> 123456789 </core:nationalNumber> </core:getPerson></pre>	Body specific for the request

Note: It's very important to respect the data set code: **_soapenv**, **_core**, **_header** and **_body** (case sensitive). If there is no header, just add a comment to avoid empty data!

For the name of the dataset header, there is no restriction, you can use whatever you want!

Technically, the information will be used like that:

```
const soapEnvelope = `  
  <soapenv:Envelope xmlns:soapenv="${soapData.soapenv}"  
    xmlns:core="${soapData.core}">  
    <soapenv:Header>  
      <core:dabsHeader>  
        ${soapData.header}  
      </core:dabsHeader>  
    </soapenv:Header>  
    <soapenv:Body>  
      ${soapData.body}  
    </soapenv:Body>  
  </soapenv:Envelope>`;  
  
console.log('soapEnvelope', soapEnvelope)  
  
const response = await fetch(url, {  
  method: 'POST',  
  headers: {  
    "Content-Type": "text/xml;charset=UTF-8",  
    "SOAPAction": ""  
  },  
  body: soapEnvelope  
});
```

Advanced functions



Function: **callScenario**

Objectives

Method to execute a scenario based on its id

Parameter(s)

Scenario ID	Id of the scenario (visible in the detail of the scenario)
--------------------	--

Example(s):

callScenario	126	Execute the tests of the scenario 126
---------------------	-----	---------------------------------------

Function: **callSuite**

Objectives

Method to execute a suite based on its id

Parameter(s)

Suite ID	Id of the suite (visible in the detail of the suite header)
-----------------	---

Example(s):

callSuite	25	Execute the tests of the suite 25
------------------	----	-----------------------------------

Function: **startTimer**

Objectives

Start a timer to measure a performance

Parameter(s)

Topic	A short name to identify the timer
--------------	------------------------------------

Example(s):

startTimer	login	Start a timer to measure the performance of the login
-------------------	-------	---

Function: **stopTimer**

Objectives

Stop a timer and store the elapsed time in the database.

Note: The timer is global to an application (not specific to a user)

Parameter(s)

Environment	Name of the environment
Topic	The identifier of the timer (must be the same name as in the startTimer)

Example(s):

stopTimer	PROD	login	Store the elapsed time in the database
------------------	------	-------	--

At the end of the execution, a variable is automatically created with a rounded value (to the second(s)).

The name of the variable is \$Timer<Topic> (topic with the first letter in upper case)

Example: \$TimerLogin.

This variable can be used in the speak function to announce a text like that:

"Execution successfully executed in \$Timer<Topic> seconds"

Note: See also the chapter on the performance in the Designer manual

Function: **promptAI****Objectives**

Thanks to the library @zerostep, we can use a ChatGPT prompt to request something.

Unfortunately, this function is not working in a REST API environment to request testing operation like 'click on the submit button'

Parameter(s)

Prompt	A sentence to request something to chatGPT
Variable	Name of the variable to store the result

Example(s):

promptAI	Generate a realistic first name for a French people	\$Name	ChatGPT prompt
-----------------	---	--------	----------------

Note: The library is free for only 500 requests per month (after, it's 40\$ per month)

This function is experimental and cannot be used for production.

In June 2025: I sent a email to the @zerostep team to ask for a solution with a REST API.

Until now, no answer received!

Mobile functions



Function: **phoneConnect**

Objectives

Establish a connection with a mobile device

Parameter(s)

--	--

Note: Please, first, read how to configure Android Studio in the Design User Manual

Function: **phoneTap**

Objectives

Simulate the tap on the screen of a mobile device

Parameter(s)

Type:	Type of type: text: Text, element: Element, web: Web page
Element	Name of the element on the mobile (see also Mobile scan in the Designer User Manual)

Example(s):

phoneTap	text	Google	Tap on the Google icon on the screen
-----------------	------	--------	--------------------------------------

Note: The library is free for only 500 requests per month (after, it's 40\$ per month)

Function: **phoneFill**

Objectives

Simulate the fill of an element on the phone

Parameter(s)

Type:	Type of type: phone : Phone, web : Web page
Element	Name of the element on the mobile (see also Mobile scan in the Designer User Manual)
Value	Value to enter

Example(s):

phoneFill	phone	com.android.chrome:id/url_bar	https://amazon.com
phoneFill	web	input[name="k"]	usb cable

Note: the phone and the web use two different methods to fill a value.

Be careful, the web element can be different from a webpage running on a desktop browser!

There is ,o **Enter** key at the end of the fill, so use the function phonePress() to do that

Function: **phonePress**

Objectives

Simulate the fill of an element on the phone

Parameter(s)

Type:	Type of type: phone : Phone, web : Web page
Element	Name of the element on the mobile (see also Mobile scan in the Designer User Manual)
Key	Enter, Home, Tab, Next, Previous, Home

Example(s):

phoneFill	phone	com.android.chrome:id/url_bar	Enter
phoneFill	phone	<N/A>	Home
phoneFill	web	input[name="k"]	Enter

Note: the phone and the web use two different methods to fill a value.

For the **Home** key on the phone, you don't need to specify a element

Function: **phoneUrl**

Objectives

Simulate open of a web page on the browser (Chrome)

Parameter(s)

Url:	Link to the web page
-------------	----------------------

Example(s):

phoneUrl	https://amazon.com	Go to Amazon.com
-----------------	--------------------	------------------

Function: **phoneCapture**

Objectives

Method to take a print screen and store the image on a slot (up to 5 slots available)

Parameter(s)

Slot	From 1 to 5
Capture mode	0 : Normal view, 1 : Full page

Example(s):

phoneCapture	1	0	Take a normal print screen and store it in the slot 1
phoneCapture	1	1	Take a full print screen and store it in the slot 1

ODBC Database functions



Function: ODBC_Generic

Objectives

Store a generic ODBC Connection string.

See also the Designer User Manual for the installation of the ODBC drivers.

Parameter(s)

connectString	The connection string delimited by ;
----------------------	--------------------------------------

Example(s):

ODBC_Generic	Driver={ODBC Driver 17 for SQL Server}; Server=server_name,1433; Database=database_name; Uid=username; Pwd=password;	Example of connect string for SQL Server. For the clarity of the string, we added carriage returns but this must be omitted in the parameter
---------------------	--	---

Note: you can use the function ODBC_Generic or a more specialized function like ODBC_SQLServer

The screenshot displays the configuration for the ODBC_Generic function. It consists of three main sections:

- Function:** A dropdown menu showing 'ODBC_Generic'.
- Connection string:** A text input field containing the string: 'Driver={ODBC Driver 17 for SQL Server}; Server=server_name,1433; Data...'. To the right of the field is a database icon.
- Status:** A dropdown menu showing 'Active'.

Function: ODBC_SQLServer

Objectives

Store a SQL Server ODBC Connection string.

See also the Designer User Manual for the installation of the ODBC drivers.

Parameter(s)

Driver	Name of the driver (be sure that the default corresponds to your configuration)
Server	Name of the server
Database	Name of the database
DummyUser	Name of the dummy user
Encrypy	yes / no (by default)

Example(s):

ODBC_SQLServer	Driver=ODBC Driver 18 for SQL Server Server=server_name,1433 Database=database_name DummyUser=Phil Encrypt=no	Example of connect string for SQL Server. DummyUser must be provided to get the login and the password (crypted or not) Note: dummy user can be <ME>
-----------------------	---	--

The screenshot shows a configuration window for the ODBC_SQLServer function. It contains the following fields and values:

- Function:** ODBC_SQLServer (selected from a dropdown menu)
- Driver:** ODBC Driver 18 for SQL Server
- Server:** localhost,143
- Database:** myDB
- Dummy User:** goffipl (with a database icon on the right)
- Encrypt:** no
- Status:** Active (selected from a dropdown menu)

Function: ODBC_Oracle

Objectives

Store an Oracle ODBC Connection string.

See also the Designer User Manual for the installation of the ODBC drivers.

Parameter(s)

Driver	Name of the driver (be sure that the default corresponds to your configuration)
Server	Name of the server
Database	Name of the database
DummyUser	Name of the dummy user

Example(s):

ODBC_Oracle	Driver=Oracle in OraClient19Home1 Server=server_name,1433 Database=database_name DummyUser=Phil	Example of connect string for Oracle. DummyUser must be provided to get the login and the password (crypted or not) Note: dummy user can be <ME>
--------------------	--	--

The screenshot displays the configuration interface for the ODBC_Oracle function. It includes a dropdown menu for the function name, followed by input fields for Driver, Server, Database, and Dummy User, and a Status dropdown. A database icon is next to the Dummy User field.

Function: ODBC_Oracle

Driver: Oracle in OraClient19Home1

Server: localhost , 1521

Database: TEST

Dummy User: goffipl

Status: Active

Function: ODBC_MySQL

Objectives

Store a MySQL ODBC Connection string.

See also the Designer User Manual for the installation of the ODBC drivers.

Parameter(s)

Driver	Name of the driver (be sure that the default corresponds to your configuration)
Server	Name of the server
Database	Name of the database
DummyUser	Name of the dummy user

Example(s):

ODBC_MySQL	Driver= MySQL ODBC 9.7 ANSI Driver Server=localhost Database=database_name DummyUser=root	Example of connect string for MySQL. DummyUser must be provided to get the login and the password (crypted or not) Note: dummy user can be <ME>
-------------------	--	---

The screenshot shows a configuration window for the ODBC_MySQL function. It contains five input fields with labels and a status dropdown:

- Function:** ODBC_MySQL (selected from a dropdown menu)
- Driver:** MySQL ODBC 9.7 ANSI Driver
- Server:** localhost
- Database:** robot
- Dummy User:** root (with a database icon on the right)
- Status:** Active (selected from a dropdown menu)

Function: ODBC_ExecuteSQL

Objectives

Execute a SQL statement with an ODBC Connection

Prior to the execution, you need to define first a connection string with the database.

The function can be used for a select as well as for DML operation (insert, delete...)

Parameter(s)

Driver	Name of the driver (be sure that the default corresponds to your configuration)
SQL	Name of the server with ? for each parameter
Parameters	Value of the parameter(s) separated by ;

Example(s):

ODBC_ExecuteSQL	SQL: SELECT * FROM `user` WHERE login = ? Parameter: goffipl	Select
ODBC_ExecuteSQL	SQL: INSERT INTO `todo` (`todo`, `position`) VALUES (?, ?) Parameter: Test;4	DML Insert
ODBC_ExecuteSQL	SQL: DELETE FROM `todo` WHERE todo= ? Parameter: Test	DML Delete

Note:

- The variable \$Error is set to 1 in case of error (otherwise 0)
- The variable \$ODBCRow contains the number of rows (Select) or the affected rows (DML)
- In case of a select, the result is stored in an internal storage (Robot database) that can be used by the function ODBC_Datavalue().

This information is persistent by user, so you can share the data between scenarios.

Function: ODBC_DataCount

Objectives

Extract the number of a record of a previous SELECT.

Prior to the execution, you need to execute a Select statement.

Parameter(s)

Variable	Name of the variable to store the number of records (starts with \$)
-----------------	--

Example(s):

ODBC_DataCount	\$Count	Store the number of records of the last select statement into the variable \$Count
-----------------------	---------	--

Note: as the select information is persistent by user, you can use the function in different scenarios.

Function: ODBC_DataValue

Objectives

Extract data of a column of a previous SELECT.

Prior to the extract, you need to execute a Select statement.

Parameter(s)

Column	Name of the column to extract the data
Row	Row of the record (> 0 and <= DataCount)
variable	Name of the variable to store the data (starts with \$)

Example(s):

ODBC_DataValue	Column: lastName Row: 1 Variable: \$Value	Store the last name of the first record into the variable \$Value
-----------------------	---	---

Note:

- The variable \$Error is set to 1 in case of error (otherwise 0)
- as the select information is persistent by user, you can use the function in different scenarios.
- If the result is undefined, check the name of the column. The variable \$Error will be set but no fatal error will be thrown!